DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON

PROJECT CHARTER
CSE 4316: SENIOR DESIGN I
Spring 2026

AGRIHOME
PRODUCT NAME

MUHAMMAD MEMON
ALEXANDER TABLIZO
SAGAR KHADKA
Govinda Awasthi
Laxman Bhushal
Nathan Tran
Thuc Dang

Revision History

Revision	Date	Author(s)	Description
0.1	02.18.2026	MM	document creation
0.2			complete draft
0.3			release candidate 1
1.0			official release
1.1			added customer change requests

CONTENTS

1 Problem Statement **6**

2 Methodology **6**

3 Value Proposition **6**

4 Development Milestones **6**

5 Background **7**

6 Related Work **7**

7 System Overview **7**

8 Roles & Responsibilities **7**

9 Cost Proposal **7**
9.1 Preliminary Budget 7
9.2 Current & Pending Support 7

10 Facilities & Equipment **8**

11 Assumptions **8**

12 Constraints **8**

13 Risks **9**

14 Documentation & Reporting **9**
14.1 Major Documentation Deliverables 9
14.1.1 Project Charter 9
14.1.2 System Requirements Specification 9
14.1.3 Architectural Design Specification 9
14.1.4 Detailed Design Specification 9
14.2 Recurring Sprint Items 9
14.2.1 Product Backlog 10
14.2.2 Sprint Planning 10
14.2.3 Sprint Goal 10
14.2.4 Sprint Backlog 10
14.2.5 Task Breakdown 10
14.2.6 Sprint Burn Down Charts 10
14.2.7 Sprint Retrospective 10
14.2.8 Individual Status Reports 10
14.2.9 Engineering Notebooks 11
14.3 Closeout Materials 11
14.3.1 System Prototype 11
14.3.2 Project Poster 11
14.3.3 Web Page 11

14.3.4 Demo Video 11
14.3.5 Source Code 11
14.3.6 Source Code Documentation 11
14.3.7 Hardware Schematics 11
14.3.8 CAD files 11
14.3.9 Installation Scripts 12
14.3.10 User Manual 12

List of Figures

1 Example sprint burn down chart 10

1 Problem Statement

Plant health management remains a significant challenge for farmers, gardeners, and greenhouse operators. Plant disease often goes undetected in early stages as symptoms may be unseen, mistaken for deficiencies, or environmental stress. Accurate diagnosis usually requires immense knowledge and because of that many individuals are unable to identify plant health issues efficiently and effectively. Incorrect identification of plant disease may result in the spread of the infection, reduce overall crop yield, and financial loss.

There is a need for a reliable system that provides early detection and continuous observation of plant health. Providing accurate disease identification would allow users to respond more effectively and efficiently which would encourage a healthy environment, minimizing crop damage, and other losses. Addressing this issue would improve plant health management while supporting more informed decision-making in not only large-scale environments but also small-scale home gardening environments.

2 Methodology

Our project is a plant disease monitoring system built with an app designed to continuously observe plant health and identify diseases at an early stage. The system will capture images of plants on a regular basis and analyze them to detect signs of any irregularities.

Using image recognition and machine learning techniques, the system will evaluate plant conditions and find potential issues based on visual symptoms. When an irregularity is detected, the system will notify the user in the app and provide information about the issue(s) found. The monitoring process will be continuous, allowing the plants to be monitored over time instead of relying on manual inspections.

The goal of this system is to provide users with an accessible and automated tool that informs and supports early detection of plant disease. By delivering timely feedback, the system helps users respond quickly to potential issues and take correct action before conditions worsen.

3 Value Proposition

The plant disease monitoring system offers significant value to stakeholders by addressing a critical need in agriculture efficiency and technological innovation. For industry sponsors in agriculture, this project represents an opportunity to participate in the development of a scalable solution that will improve early plant diseases and reduce crop loss. Early identification of plant health issues can reduce financial risk and improve overall quality of the plants.

For the university and academic stakeholders, this project strengthens the versatility of knowledge by integrating computer science, engineering, and agricultural applications. It improves the institutions commitment and to solving real world issues. Successful development of this system can also serve as future research initiatives and grant proposals from agricultural organizations.

By investing funding, time, and expertise into this project, stakeholders are supporting innovation in sustainable agriculture, advancing applied research, and contributing to a solution with meaningful economic and environmental impact.

4 Development Milestones

- Project Charter first draft – February 2026
- System Requirements Specification - March 2026
- Architectural Design Specification – April 2026

- Demonstration of AI detection and Test Rig - April 2026
- Detailed Design Specification - TBA
- Demonstration of AI improvements and User Interface - TBA
- Demonstration of Web server and AI integration - TBA
- CoE Innovation Day poster presentation - TBA
- Final Project Demonstration - TBA

5 BACKGROUND

Plant health monitoring remains a persistent challenge across home gardening, greenhouse cultivation, and small- to medium-scale agricultural operations. Many plant species are susceptible to diseases, pest infestations, and nutrient deficiencies that initially manifest as subtle visual symptoms such as discoloration, spotting, wilting, or irregular leaf texture. These early-stage symptoms are often misinterpreted as environmental stress or minor deficiencies, leading to delayed intervention and potentially significant crop loss.

The current status quo relies heavily on manual visual inspection and personal experience. Accurate diagnosis frequently requires expertise in plant pathology, which is not readily accessible to many growers. Even when symptoms are recognized, identifying the specific cause and determining appropriate corrective actions can be difficult. This gap between symptom recognition and actionable decision-making results in inefficient pesticide use, improper fertilization, and avoidable yield reduction.

From a business perspective, crop loss due to undetected or misdiagnosed plant health issues represents a significant economic risk. Small-scale growers and greenhouse operators may experience reduced productivity and increased input costs when issues are addressed too late or treated incorrectly. Furthermore, unnecessary chemical treatments contribute to environmental concerns and increased operational expenses. There is therefore a clear opportunity to provide an accessible, automated tool that assists users in early detection and informed intervention.

Advances in computer vision and machine learning provide a viable technological foundation for addressing this gap. Deep learning-based image recognition systems have demonstrated strong performance in plant disease classification across diverse species. In addition, image segmentation techniques allow isolation of individual leaves from complex backgrounds, improving detection accuracy. However, many existing solutions focus primarily on classification accuracy and do not provide structured guidance on next steps, limiting their practical usability.

AgriHome aims to bridge this gap by integrating leaf isolation, issue classification, and actionable recommendation into a single decision-support system. By capturing an image, isolating potentially unhealthy leaf regions, and identifying likely issues using machine learning models, the system will inform users not only of detected conditions but also of recommended corrective measures. These may include environmental adjustments, nutrient balancing suggestions, or consultation prompts when expert intervention is required.

The development team does not currently have a direct industrial sponsor; however, the system is designed to serve a broad customer base including home gardeners, greenhouse operators, and educational agricultural programs. The project also aligns with the university's emphasis on applied artificial intelligence and sustainable technology innovation. Through this effort, AgriHome seeks to demonstrate how accessible computer vision tools can enhance plant health management while reducing dependency on expert-only diagnostics.

6 RELATED WORK

Automated plant disease detection using image recognition has been widely studied in academic research. Convolutional Neural Networks (CNNs) have demonstrated strong performance in classifying plant diseases from leaf images. Mohanty et al. [1] showed that deep learning models trained on the

PlantVillage dataset could achieve high accuracy in multi-class plant disease classification tasks, including tomato leaf diseases. Their work demonstrated that transfer learning using pretrained CNN architectures significantly improves classification performance compared to traditional machine learning approaches.

Subsequent research has improved upon these models by comparing advanced architectures such as ResNet, VGG, MobileNet, and EfficientNet for tomato leaf disease detection [2], [3]. These studies report classification accuracies often exceeding 95% under controlled dataset conditions. More recent work has incorporated attention mechanisms and transformer-based models to improve fine-grained disease recognition and robustness under varying image conditions [4].

In addition to classification, several researchers have explored leaf segmentation techniques to isolate plant tissue from background noise prior to disease analysis. Methods such as U-Net and Mask R-CNN have been used to improve detection accuracy by focusing on infected leaf regions [5]. Segmentation improves model interpretability and reduces interference from complex backgrounds.

Commercial solutions also exist in the form of mobile applications and agricultural advisory platforms. Applications such as Plantix and similar plant-diagnosis tools allow users to upload plant images and receive predicted disease labels. However, many commercial tools operate as black-box systems, providing limited transparency regarding prediction confidence and model reasoning. Additionally, such tools may not provide structured next-step recommendations tailored to specific disease severity or uncertainty levels.

Despite strong academic progress, several limitations remain. Many research models are trained and evaluated on curated datasets with controlled lighting and backgrounds, which may not generalize well to real-world user-captured images. Furthermore, many systems emphasize classification accuracy but do not integrate explainable AI techniques (e.g., heatmaps) or structured recommendation logic to support user decision-making.

AgriHome differentiates itself by integrating three components into a unified system: (1) leaf isolation to reduce background noise, (2) machine learning-based disease classification with confidence scoring, and (3) actionable next-step guidance for users. By combining image recognition with decision-support functionality, the proposed system aims to improve practical usability beyond existing research prototypes and commercial black-box applications.

7 System Overview

The proposed solution is a camera-based plant monitoring system designed to observe multiple plants arranged on a rack. A wide-angle camera captures periodic images of the entire rack. These images are processed to identify plant regions (primarily visible leaves), extract basic health and growth indicators, and store results over time. The system then presents this information to the user through a simple dashboard that displays current plant status, historical trends, and alerts when abnormalities are detected.

At a high level, the system follows this flow:

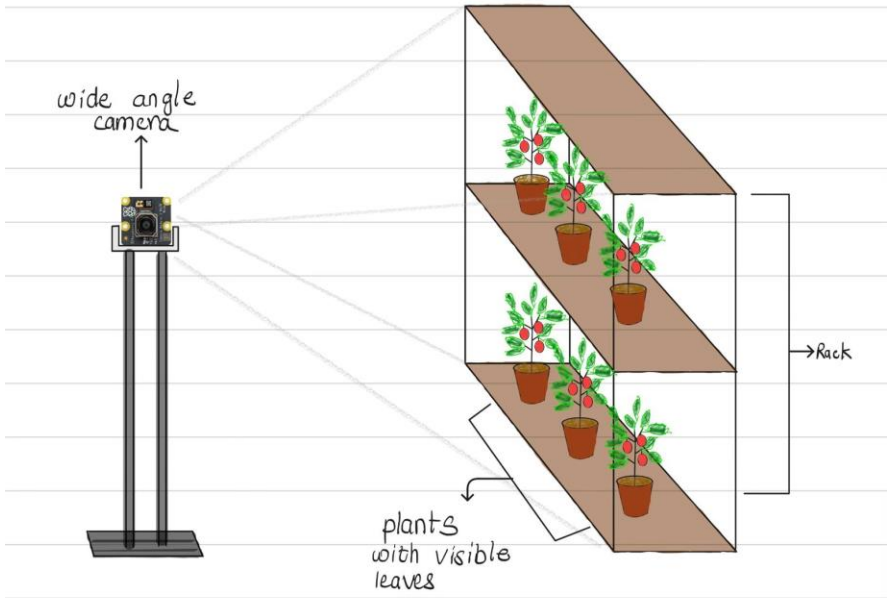
1. **Image Capture:** The camera captures time-stamped images of the rack.

2. **Rack Mapping:** The system divides the image into predefined shelf and plant zones.

3. **Plant Analysis:** Image processing detects plant areas and calculates indicators such as leaf coverage, color changes, and visible anomalies.

4. **Data Storage:** Images and measurements are stored to track trends over time.
5. **User Interface:** A dashboard displays plant health summaries, comparisons, and notifications.
6. **Optional Integration:** External systems (e.g., irrigation or environmental logs) may provide event data to correlate with plant changes.

```
+------------------+
|     User      |
| Dashboard/View |
+---------+--------+
       |
       v
+---------------------+
| Dashboard & Alerts  |
+----------+-----------+
       |
       v
+---------------------+
| Data Storage       |
| (images + metrics)  |
+----------+-----------+
       ^
       |
+---------------------+
| Plant Analysis      |
| (detection + scoring)|
+----------+-----------+
       ^
       |
+---------------------+
| Rack Mapping       |
```



```
| (shelf/plant zones)  |
+----------+-----------+
        ^
        |
+----------------------+
| Image Capture       |
| (wide-angle camera)  |
+----------------------+
```

8 Roles & Responsibilities

Stakeholders for the project are those who want a compact plant monitor that can detect diseases in plants. The sponsor for this project is Professor Noor.

There are currently 4 key roles in this team. Roles include the scrum master, the hardware designers/programmers, the software programmers, and AI designers. The scrum master handles the task list and acts as the main point of contact for the project's sponser. The hardware designers role involves the contrustion and maintainance of the plant rack, as well as the programming the firmware and drivers for the hardware componets. The software programmers focus of the user interface, web server, and AI implementation. The AI designers will train and test the AI, as well as work with the rest of the team for integration.

All roles are not concrete, and no person is tasked with one for the duration of product development. All roles are expected to change at the end of each sprint, and the responsibility of each role is subject to change. Members of the team are also able to take on multiple roles. The scrum master in particular is expected to take on the responsibilities one of the other roles as well.

Roles and tasks will be assigned to each team member based on expertise and ability, so while no one has a permanent role, it is likely that the same people will be assigned to similar tasks and roles across each sprint. The exception to this is the scrum master, who will change at the end of each sprint at the request of Professor Noor.

9 Cost Proposal

The budget allotted by the customer is $800, which is sourced from the CSE department of UTA. This will be used to acquire necessary hardware for image capture, running AI models, and hosting the frontend webserver. The budget will also be used for system implementation, including plants, and materials for constructing a testbed.

9.1 Preliminary Budget

Item	Estimated Cost
Camera	$70
Computer (Local)	$60
Computer (Server)	$0 (Provided by UTA)
Camera Fixture	$50
Plant Fixture	$100
Plants	$50

Current & Pending Support

The primary source of funding for this project is provided by the CSE Department of The University of Texas at Arlington. The default amount stated is $800, which should be able to cover all of the projected costs. In case additional funding is required, either due to exhausting the default funding amount or being unable to use university funds to acquire necessary materials, Professor Noor has stated that he would be willing to provide additional funding if required, though an exact amount nor formal commitment has been made.

10 FACILITIES & EQUIPMENT

Most of the equipment needed to complete this project is available at UTA. Construction of our plant rack and camera fixture will utilize the makerspaces at UTA, specifically the ones located in Netherman Hall and the Library. The tools required will include saws, and 3D printers, but other tools such as CNC milling or laser cutting may be used should the design of the camera fixture require it.

Our AI model will require a computer with sufficient GPU or AI processing power. This is for both training and deployment. Computers in the Senior Design lab will have the capability, but there are limits to availability. If lab computers are not available, members of the team should have machines with the compute needed to train and run AI models as well. If no other options are available, renting an external server will be considered, but for the scope of the project proposed, it shouldn't be needed.

For plant monitoring and care, a stable temperature and electrical supply is required. The Senior Design lab has enough space to store the camera fixture and plant rack, however due to the significant amount of space the testbed may take up, storing it outside on one of the balconies in the Engineering Research building has been considered. Optional requirements may include sufficient sunlight for our test plants, as well as a stable temperature within the range of 68 F to 78 F. If the optional requirements cannot be meet due to weather, space, or convivence, the test plants may be relocated to a space that meets those requirements. Other spaces may be considered besides the ones listed if such options are made available.

Testing the monitoring system does not require any additional space or considerations; however a lab computer in proximity will make the process easier. The ideal space for testing would be the Senior Design lab, but other spaces such as one of the balconies in the Engineering Research building may be used. Other options for test sights may be considered provided they meet the requirements necessary for storage and environmental compatibility with our test plants.

11 ASSUMPTIONS

The following list contains critical assumptions related to the implementation and testing of the tomato disease detection system:

• A sufficient number of labeled tomato leaf disease images will be available for training and testing the machine learning model.

• Access to suitable image capture equipment (camera and fixture) will be available by the second sprint cycle.

• Team members will have consistent access to computing resources capable of training and running machine learning models.

• The development team will be granted access to required university lab spaces during normal operating hours for system setup and testing.

• The image recognition model will achieve acceptable classification performance (e.g., >80% validation accuracy) within the planned training period.

• Environmental lighting conditions during image capture will be sufficiently controlled to avoid extreme variability in image quality.

• The instructor and course schedule will remain consistent with the projected sprint timeline.

12 Constraints

The following list contains key constraints related to the implementation and testing of the project:

• The final prototype demonstration must be completed by the end of Spring 2026.

• Total development costs must not exceed the allocated $800 budget.

• Development must follow the sprint schedule defined by the course syllabus.

• The system must operate using standard consumer-grade cameras and computing hardware.

• The solution must comply with university IT and lab usage policies.

• Only publicly available datasets or self-collected images may be used for model training.

• The team size and available development hours are fixed by course enrollment and academic commitments.

13 RISKS

he following high-level risk census contains identified project risks with the highest exposure. Risk exposure is calculated as:

Exposure = Probability × Loss (in days)

Risk description	Probability	Loss (days)	Exposure (days)
Insufficient model accuracy for reliable diagnosis	0.40	15	6.0
Limited availability of labeled tomato disease images	0.35	12	4.2
Hardware malfunction or camera failure	0.25	8	2.0
Unexpected delays in model training or debugging	0.30	10	3.0
Team member availability conflicts	0.30	7	2.1
Image quality variability causing inconsistent results	0.35	9	3.15

Table 1: Overview of highest exposure project risks

Mitigation strategies (to be detailed later) include early dataset validation, backup hardware planning, and incremental model testing per sprint.

14 DOCUMENTATION & REPORTING

14.1 MAJOR DOCUMENTATION DELIVERABLES

The Project Charter defines project scope, objectives, stakeholders, and high-level risks.
It will be reviewed at major milestone checkpoints and updated only if scope, schedule, or risk changes occur.

Initial Version Delivery: Sprint 1

Final Version Delivery: Prior to final project submission

14.1.1 Project Charter

The Project Charter will be maintained as a controlled document that defines the project's vision, scope, objectives, stakeholders, and high-level risks. It will be reviewed at the end of major milestones or sprints to ensure alignment with the project direction. Updates will only occur if there are significant scope changes, stakeholder changes, or major risk adjustments. Minor wording or formatting edits may be made as needed. All major revisions will be version-tracked and approved by the team.

Initial Version Delivery:
The initial version will be delivered during the first sprint after initial planning discussions.

Final Version Delivery:
The final version will be delivered at the end of the project prior to final submission, reflecting any approved updates made during development.

14.1.2 System Requirements Specification

The System Requirements Specification (SRS) will be maintained as a living document throughout the project to ensure it accurately reflects agreed-upon system functionality and constraints.

Maintenance and Updates

- The SRS will be formally reviewed at the beginning and end of each sprint.
- Updates will occur whenever new requirements are approved, existing requirements are
- modified, or clarifications are needed.
- Changes may result from customer feedback, sprint reviews, testing outcomes, or scope adjustments.
- Minor edits (clarifications, wording, formatting) may be made as needed.
- Major requirement changes must be discussed with the team and approved before being incorporated, with version tracking to document revisions.

This process ensures that development remains aligned with documented requirements and that the SRS reflects the most current and validated system expectations.

Initial Version Delivery
The initial version of the SRS will be delivered early in the project, typically during the first or second sprint after requirements elicitation and stakeholder discussions are completed.

Final Version Delivery
The final version of the SRS will be delivered at the end of the project prior to final submission. It will reflect all approved changes and represent the finalized set of validated system requirements.

14.1.3 Architectural Design Specification

The Architectural Design Specification describes the high-level system structure, major components, interfaces, and data flow between subsystems.

Maintenance and Updates

This document will be reviewed at the end of each sprint to ensure the architecture remains aligned with the implemented system. Updates will occur if there are significant architectural changes, such as adding new subsystems, modifying component interactions, or changing integration approaches. Minor refinements or clarifications may be made as needed. All major revisions will be version-controlled and agreed upon by the team before being finalized.

Initial Version Delivery

The initial version will be delivered after the System Requirements Specification is completed and approved, typically during an early design sprint before full implementation begins.

Final Version Delivery

The final version will be delivered at the end of the project prior to final submission, reflecting the validated architecture of the completed system.

14.1.4 Detailed Design Specification

The Detailed Design Specification describes the internal design of individual modules, workflows, data structures, and algorithms required to implement the system.

Maintenance and Updates

This document will be updated continuously throughout development as features are implemented and refined. It will be reviewed at the end of each sprint to ensure it accurately reflects the current implementation. Updates will occur when module logic changes, new components are added, interfaces are modified, or improvements are made based on testing and feedback. Minor clarifications may be made as needed, while significant design changes will be version-tracked and approved by the team.

Initial Version Delivery

The initial version will be delivered after the architectural design is completed, typically during the early implementation phase of the project.

Final Version Delivery

The final version will be delivered at the end of the project prior to final submission, fully reflecting the completed and validated system design.

14.2 Recurring Sprint Items

14.2.1 Product Backlog

Product backlog items will be derived directly from the System Requirements Specification (SRS). Each functional and non-functional requirement in the SRS will be broken down into smaller, actionable backlog items such as user stories, features, enhancements, or technical tasks. If new requirements arise from instructor feedback, testing, or design changes, they will also be added to the backlog after team discussion and approval.

Prioritization will be based on academic deadlines, project milestones, technical dependencies, and overall impact on the system. Items required for major deliverables (e.g., documentation, core features) will receive higher priority. The Scrum Master for the current sprint will facilitate prioritization discussions, but decisions will be made collaboratively by the team to ensure agreement and feasibility.

The product backlog will be maintained using a shared project management tool (such as Jira, Trello, or a similar collaboration platform). This tool will allow all team members to view, update, and track backlog items. It will also provide transparency for instructors or stakeholders if review access is needed.

14.2.2 Sprint Planning

Each sprint will begin with a formal sprint planning meeting. The Scrum Master for that sprint will facilitate the meeting. During planning, the team will:

1. Review the product backlog and current project progress.
2. Consider upcoming course deadlines and required deliverables.
3. Define the sprint goal.
4. Select high-priority backlog items that align with the sprint goal and team capacity.
5. Break selected items into smaller tasks and estimate effort.

The team will ensure that the selected work is realistic and achievable within the sprint timeframe before committing it.

Based on typical Senior Design II course schedules and semester timelines, the project is expected to have approximately **5–7 sprints**, depending on the length of the semester and major milestone deadlines. The exact number of sprints will align with the official course schedule and submission dates provided by the instructor.

14.2.3 Sprint Goal

For this college project, the sprint goal will be decided by the Scrum Master assigned for that specific sprint. Since the Scrum Master role rotates each sprint, the responsibility of proposing and guiding the sprint goal discussion will also rotate among team members.

At the beginning of each sprint, the current Scrum Master will review the product backlog, course deadlines, and upcoming deliverables. They will then propose a sprint goal that aligns with project priorities and academic requirements. The full team will discuss and agree on the goal to ensure it is realistic and achievable within the sprint timeframe.

Customer involvement (instructor or TA) will occur through reviewing assignment instructions, grading rubrics, and feedback from previous submissions. This ensures that each sprint goal aligns with course expectations and contributes directly to the required deliverables.

14.2.4 Sprint Backlog

The sprint backlog will be determined during the sprint planning meeting. The Scrum Master for that sprint will facilitate the discussion, but the entire team will collaboratively decide which product backlog items are selected. Selection will be based on priority, task dependencies, team capacity, and the defined sprint goal. The team must agree that the chosen items can realistically be completed within the sprint timeframe.

The sprint backlog will be maintained using a shared collaboration tool (such as Jira, Trello, or a similar project management platform). A digital scrum board will be used with columns such as To Do, In Progress, Testing, and Done. Team members will update task status regularly to ensure transparency, track progress, and support daily stand-up meetings.

14.2.5 Task Breakdown

Once items are selected for the sprint backlog, they will be broken down into smaller, manageable tasks during sprint planning. The Scrum Master for that sprint will facilitate this breakdown process, ensuring tasks are clear and actionable.

Task assignments will primarily be voluntary. Team members will claim tasks based on their skills, experience, and current workload. This encourages accountability and ownership. However, if certain tasks remain unclaimed or workload becomes unbalanced, the Scrum Master may help redistribute tasks to ensure fairness and timely completion.

Time spent on tasks will be documented using the team's collaboration tool (e.g., Jira, Trello, or similar). Team members will update task status regularly and provide time estimates or actual hours worked. Progress will also be discussed during daily stand-up meetings to maintain transparency and track completion toward the sprint goal.

14.2.6 Sprint Burn Down Charts

The scrum master is tasked with creating the burndown chart. All hours are logged using Jira and reports are generated using the report tool at the end of each sprint.

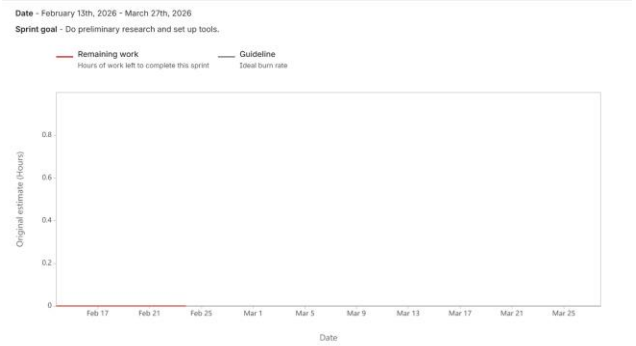


Figure 1: Example sprint burn down chart

14.2.7 Sprint Retrospective

The sprint retrospective will be conducted as a team discussion at the end of each sprint, after the sprint review and before the next sprint planning meeting. The Scrum Master for that sprint will facilitate the meeting to ensure all team members have the opportunity to provide input.

During the retrospective, the team will discuss:

- What went well during the sprint
- What did not go well
- Challenges encountered
- What can be improved in the next sprint
- Whether the sprint goal was achieved

As a group, we will document key discussion points, agreed-upon improvements, and specific action items for the next sprint. This documentation will be recorded in our shared collaboration tool or project folder.

Individually, each team member may document personal reflections, including their contributions, challenges faced, and areas for improvement. Individual reflections will be submitted according to course requirements, typically shortly after the sprint ends.

The group retrospective summary will be completed and documented within a few days after the sprint concludes and before the next sprint officially begins.

14.2.8 Individual Status Reports

Each team member will write a report detailing the tasks done for during the sprint, the hours spent on each task, and tasks they plan on doing during the next sprint. This report will be submitted between the end of the current sprint and the start of the next sprint.

14.2.9 Engineering Notebooks

Engineering notebooks will be updated based on the type of work done by each member. In general, each member is expected to document all changes made to the project. There is no minimum number of pages to be completed each sprint. All changes and documentation done to a member's engineering notebook will be shared with the rest of the team during the start of the work week. Whenever a witness is required to sign a document, it is expected that other members working on the same or similar task will provide a signature.

14.3 Closeout Materials

14.3.1 System Prototype

The final prototype will consist of a plant rack, a camera fixture, a local computer, and a remote server. A demonstration will include a showcase of the user interface, control over the plant rack and its monitoring capabilities, the ability of the camera fixture to capture images of the plants over a set period, and a test of the AI model to detect diseases.

No PAT is required by the customer currently. No off-site test is required for this project, as all features should be able to be demonstrated within the demo lab.

14.3.2 Project Poster

No details on a project poster and its requirements have been released.

14.3.3 Web Page

There are no plans to create a public web page at this time.

14.3.4 Demo Video

A demo video will showcase the user interface, and the ability of the AI to detect diseases. No plans have been made for the demo video, but a proper showcase, explanation, and demonstration should take approximately 10 minutes.

14.3.5 Source Code

All source code will be stored in a git repository. No requests for source code have been made by the customer currently. If code is requested, we will provide access to the repository and instructions to build the programs should they be needed.

14.3.6 Source Code Documentation

Documentation will follow agile documenting best practices. All documentation will be done with Doxygen and saved as a PDF.

14.3.7 Hardware Schematics

A wiring diagram with connections, and communication protocols for the plant rack's lighting, camera, motors, and sensors will created.

14.3.8 CAD files

CAD files for 3D printed components and the overall plant rack's design will be made using Fusion. Both a STEP file, and STLs for 3D printed components will be posted in the git repostitory.

14.3.9 Installation Scripts

The user is only expected to access a web application, as all programs are preinstalled on the local computer. No installation script will be required unless the customer requests one in the future.

14.3.10 User Manual

A user manual will be provided as a digital pdf. The instructions will include setup of the plant rack, installation of the camera fixture, and ideal conditions for growth.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, 2016.

[2] H. Durmuş, E. O. Güneş, and M. Kırcı, "Disease detection on the leaves of the tomato plants by using deep learning," *2017 6th International Conference on Agro-Geoinformatics*.

[3] A. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, 2018.

[4] Recent transformer-based tomato disease classification research (e.g., attention-based vision models in agricultural image classification, 2023–2025 publications).

[5] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI*, 2015.